



UNIVERSIDAD DEL  
ACONCAGUA

# Métodos ágiles de desarrollo de software

## Programación Extrema (XP)



Diseño II  
Prof. Pablo Sánchez





UNIVERSIDAD DEL  
ACONCAGUA

# Valores de XP

Como la mayoría de los métodos ágiles, XP está basada en **VALORES**:

## Comunicación

Comunicación **estrecha pero verbal**, cara a cara. **Metáforas** para conceptos importantes.



## Retroalimentación

Debe funcionar en forma **permanente**, por parte del cliente y de los desarrolladores.

## Simplicidad

Hacer **sólo lo que se necesita en el momento, no más**. Valor esencial, que maximiza la inversión actual.

## Respeto

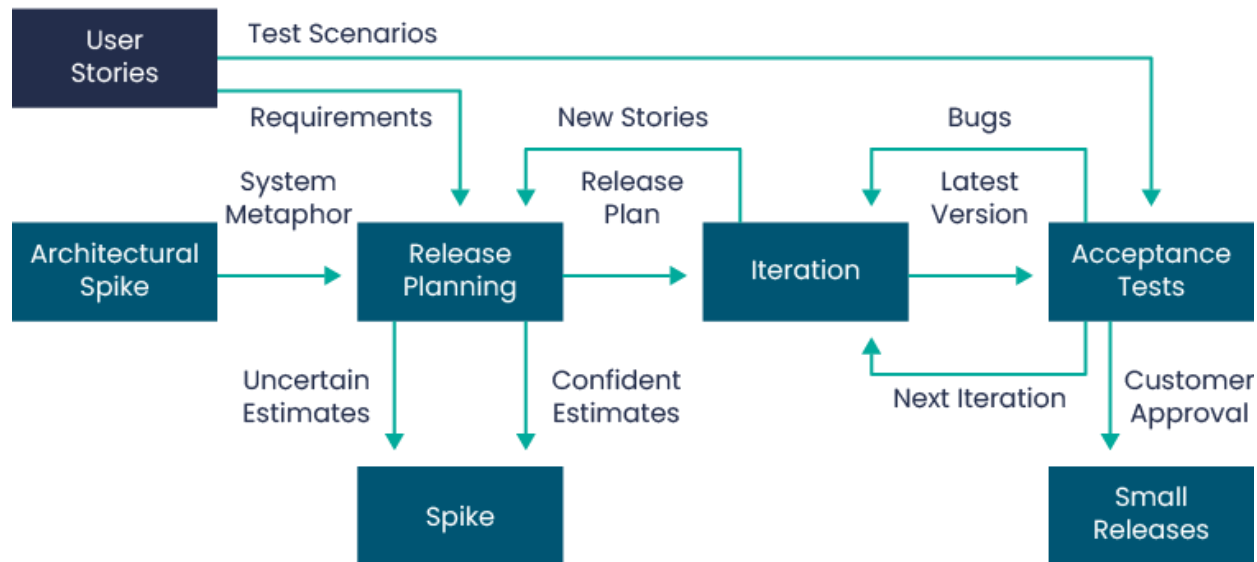
Entre los desarrolladores, clientes y managers. Todos aportan valor y know how al proyecto

## Coraje (Valor)

Muchas prácticas XP requieren disciplina y valor (asumir problemas, cambios, no anticiparse, etc).

# Ciclo de vida de XP

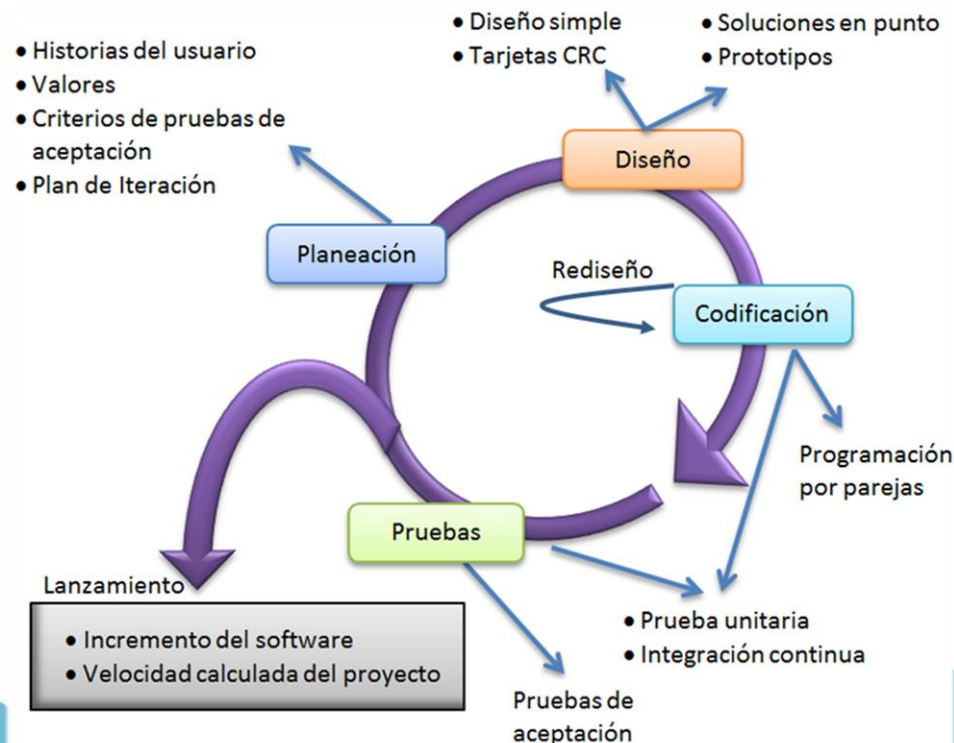
El ciclo de vida de XP pone énfasis en los aspectos técnicos y las prácticas específicas del desarrollo de software.



**Roles:** *La estructura de roles en XP es intencionalmente plana y colaborativa. Cliente, Desarrollador, Coach, Tracker y Manager son roles habituales.*

# El proceso XP: Reglas y Prácticas

En la visión de Kent Beck, XP está diseñado como un conjunto de **REGLAS y PRÁCTICAS** que se aplican en el contexto de 4 actividades estructurales: **Planeación, Diseño, Codificación y Pruebas**, además de prácticas relacionadas a la **Gestión del Proyecto**:



<http://www.extremeprogramming.org/>



# Reglas y prácticas: Planeación

- ✓ **Historias de usuario:** El proyecto comienza recopilando historias de usuario, escritas por el cliente en su propio lenguaje. Son descripciones cortas de lo que el sistema debe hacer, con un nivel de detalle mínimo, suficiente para que los desarrolladores estimen el tiempo de implementación.
- ✓ **Plan de entregas (Release Plan):** En una reunión de planeamiento" (o "planning game") donde participan todos los actores del proyecto, se establece el **orden de prioridad** en que se tomarán las HU para las entregas, y **cómo se agruparán**. Cada historia, 1 a 3 semanas de "tiempo ideal de desarrollo". De aquí surge el **cronograma de entregas**.
- ✓ **Plan de iteración:** Las iteraciones son los "latidos del proyecto", **deben mantenerse constantes**. Al principio de la iteración, se descomponen las historias seleccionadas en tareas de programación, y se establecen las pruebas de aceptación correspondientes.



UNIVERSIDAD DEL  
ACONCAGUA



# Reglas y prácticas: Diseño

- ✓ **Simplicidad:** XP propone implementar el diseño más simple que pueda funcionar, ya que un diseño simple se implementa más rápido que uno complejo. La simplicidad es un concepto subjetivo, y requiere conocimiento para reconocerla: codificar sólo las características necesarias, y luego refactorizar incrementalmente.
- ✓ **Refactorización:** Consiste en reescribir parte del código sin cambiar su funcionalidad, para hacerlo más simple, conciso o entendible. XP sugiere refactorizar sin piedad y constantemente, como una manera de hacer evolucionar el código a un diseño más simple, que favorecerá su ampliación o cambio futuro.



# Reglas y prácticas: Diseño

- ✓ **Metáfora:** Es una manera sencilla de entender el propósito del proyecto, a través de una **analogía que sea de comprensión común** para clientes y desarrolladores. Debe servir para explicar el diseño del sistema, y para nombrar clases y métodos consistentemente (ej: "línea de producción", "lista de materiales"). Difícil !!
- ✓ **Solución spike:** Prototipo para HU riesgosas (a nivel técnico o arquitectónico).
- ✓ **Tarjetas CRC:** Clase-Responsabilidad-Colaboración (producto de diseño en XP).







UNIVERSIDAD DEL  
ACONCAGUA



# Reglas y prácticas: Codificación

- ✓ **Cliente siempre disponible:** Es uno de los requisitos de XP. El cliente debe ser parte del equipo de desarrollo, ya que no hay documentos detallados de requerimientos. Además es esencial en el testing del software. Expertos !!!
- ✓ **Estándares de codificación:** Lograr que el código sea comprendido por todo el equipo, "como si hubiera sido desarrollado por una persona". Refactorizar!
- ✓ **Desarrollo dirigido por las pruebas:** En XP, los desarrolladores primero crean las pruebas unitarias y luego codifican el software que sea capaz de superar esas pruebas. → Retroalimentación inmediata y codificación simple.
- ✓ **Programación en pares:** 2 personas en la misma computadora. Incrementa la calidad, mejora la productividad y la motivación. Requiere aprendizaje.
- ✓ **Integración continua:** A medida que se termina una funcionalidad, la pareja responsable debe integrarla, a su turno. Esto debe ser a diario si es posible.
- ✓ **Propiedad colectiva del código:** Cualquier desarrollador puede modificar algo.





# Reglas y prácticas: Pruebas

- ✓ **Pruebas Unitarias:** Es una de las **pedras angulares** de XP. Deben automatizarse, para ejecutarlas repetidamente con facilidad (regresión). Permiten la propiedad colectiva y la refactorización, ya que **custodian** el código. Cualquier código debe pasar el 100 % de las pruebas antes de ser liberado o integrado.
- ✓ **Corrección de errores:** Al detectar un “bug”, debe crearse un test de aceptación (que falle!!) que demuestre el problema. Luego los programadores crearán pruebas unitarias, para después corregir el problema y probar.
- ✓ **Pruebas de Aceptación:** Son **escenarios especificados por el cliente a partir de las historias de usuario**, creadas durante la iteración en la que se implementan esas funcionalidades. No puede considerarse finalizada una HU hasta que no se superen las pruebas de aceptación en un 100%. El tiempo necesario para la corrección de errores debe preverse en la iteración. Son pruebas de “caja negra”.



# Reglas y prácticas: Gestión

- ✓ **Espacio de trabajo:** Preferentemente debería ser una **sala amplia, sin divisiones**, con los equipos para codificar en pares en el centro (escritorios “sin propiedad”), y espacios individuales en la periferia (algo de privacidad sin desconectarse). Además debería preverse espacio para las reuniones diarias, una mesa para discusiones grupales, y pizarras o paneles para anotaciones. Comunicación!!
- ✓ **Ritmo sostenido:** Planificar el trabajo de manera de mantener un **ritmo constante y razonable, sin sobrecargar al equipo**. Trabajar tiempo extra con frecuencia atenta contra la motivación, afecta el progreso e impacta en la calidad del producto. Renegociar objetivos !!
- ✓ **Reuniones diarias de pie:** XP propone comenzar el día de trabajo con reuniones breves de todo el equipo de desarrollo, de pie en círculo. La idea es comunicar problemas y soluciones, y mantener focalizado al equipo. Como mínimo debe incluirse: qué se hizo ayer, qué se hará hoy, y qué situaciones están causando retrasos.



UNIVERSIDAD DEL  
ACONCAGUA



# Reglas y prácticas: Gestión

- ✓ **Rotación de las personas:** Para evitar pérdidas de conocimiento y cuellos de botella en la codificación, XP sugiere impulsar a las personas a trabajar en nuevas secciones del sistema frecuentemente, apoyados en la programación en pares. La idea es **evitar “islas de conocimiento” y balancear las cargas de trabajo.**
- ✓ **Adaptar o reparar el proceso XP:** La **necesidad de adaptación del proceso a un proyecto específico es asumida como un hecho.** Al comienzo pueden aplicarse las reglas y prácticas como están definidas, para luego ser cambiadas cuando el equipo lo decida. Deben realizarse reuniones para analizar lo que está funcionando en el proceso y lo que debe mejorarse. Esta mejora debería ser parte del proceso de desarrollo.



# Aplicabilidad de XP



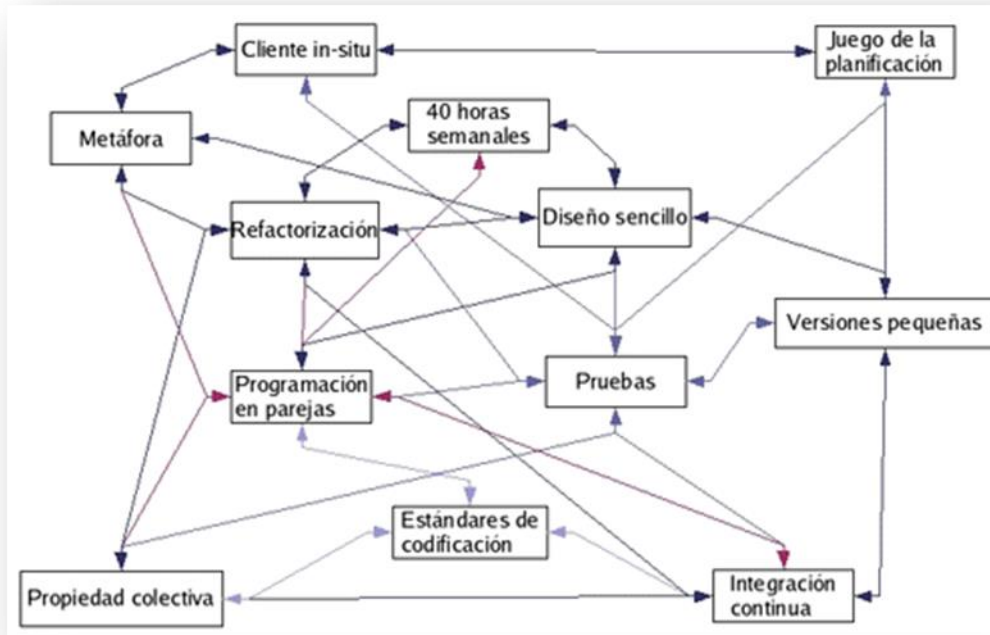
Si bien XP ha generado diferentes visiones respecto a su utilidad, hay cierto consenso en cuanto a las situaciones en las que puede utilizarse con cierto éxito:

- Proyectos donde **los requerimientos tienden a cambiar con frecuencia**, ya sea porque el dominio del problema es cambiante, o porque los clientes no tienen una idea firme de lo que debería hacer el sistema.
- Cuando hay un **nivel considerable de riesgo**, ya sea por calendarios apremiantes o ámbitos desconocidos para el equipo o para la industria.
- Proyectos con un **pequeño a mediano grupo de desarrolladores** (aprox. entre 2 y 12) y no están dispersos geográficamente. No es necesario programadores altamente calificados, pero sí en cantidades acotadas.
- Donde pueda contarse con **equipos de trabajo extendidos**, que incluya a clientes y gerentes trabajando codo a codo, para garantizar alta productividad.



# Críticas / limitaciones de XP

- ❑ Codependencia de las prácticas y reglas:



- ❑ Estrés por ausencia de feedback: Cuando los clientes no tienen tiempo, disposición o experiencia para participar en el desarrollo, el equipo puede sufrir al acercarse a los plazos establecidos.
- ❑ Otras críticas ¿?